Pvlseon

I treated the brief as more than a dashboard; the real goal was turning messy CSVs into clear KPI decisions fast.

Take-home objective

Build a working Smart KPI Dashboard that loads business data, maps columns cleanly, calculates the KPIs that matter, explains the movement with Claude, and exports a summary someone can actually review.

Data integration

CSV upload plus built-in demo datasets, with column mapping that can be adjusted when the file is not perfect.

KPI clarity

Revenue, orders, profit, margin, AOV, units, and discount shown through simple KPI cards and monthly trends.

AI commentary

Claude insights only run when requested, with a local fallback so the demo still works without an API key.

Deliverable

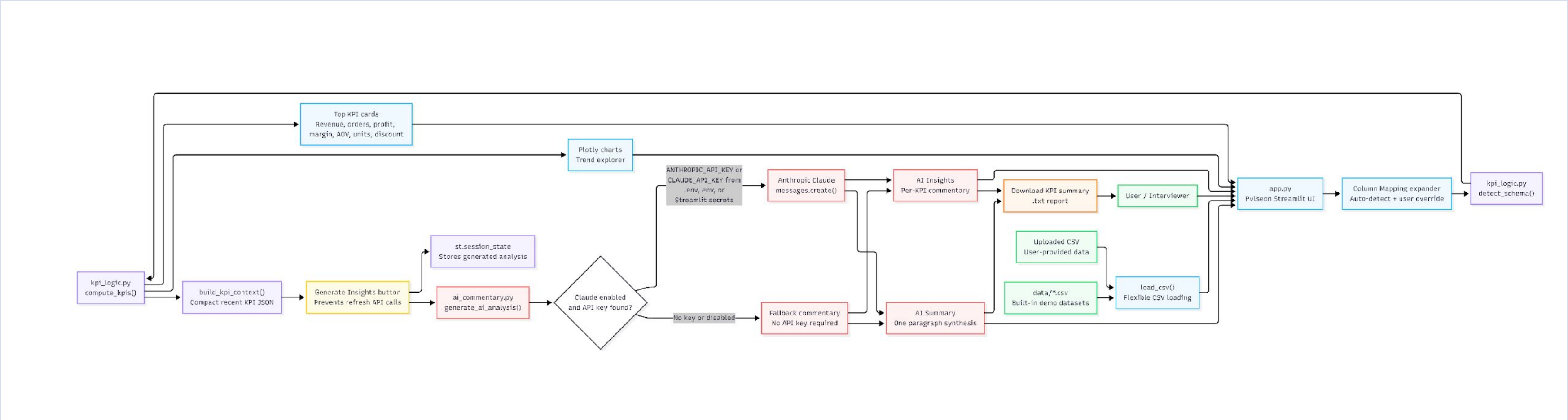
PDF export wraps the KPI cards, chart images, AI insights, and summary into one clean deliverable.

Success criteria I designed around

- Works even when the CSV is not perfect
- No surprise API calls on Streamlit reruns
- AI commentary matches the KPIs on screen
- Simple to demo; structured enough to grow

Python | Streamlit | Claude

I kept the app split into clean pieces so ingestion, KPI math, AI, and export are not tangled together.



Flexible input

Uploaded CSVs and five demo datasets go through the same load_csv path.

Schema layer

detect_schema handles the first pass, then manual overrides keep the app from being locked to one exact schema.

AI gate

Claude only runs after Generate is clicked; session state keeps the result instead of calling again on refresh.

Export

The PDF uses the same KPI values and chart figures already shown in Streamlit, so the export does not drift from the app.

The build stays intentionally focused, but the workflow decisions are what make it feel reliable instead of stitched together.

1 Map

Auto-detect columns first, then let the user fix the mapping when needed.

2 Compute

Roll up monthly KPIs in kpi_logic.py using stable internal column names.

3 Explain

Build compact KPI context, then ask Claude for business-friendly commentary.

4 Export

Render the same cards, charts, and AI text into a reviewer-ready PDF.

Button-gated AI generation

```
if st.button(generate_label):
    st.session_state["ai_analysis"] = cached_ai_analysis(
        context, model, enable_ai
    )

# no API call happens on refresh
```

AI prompt shape

Claude receives recent KPI history and returns structured JSON for revenue, orders, profit, margin, and the summary.

No-key behavior

If ANTHROPIC_API_KEY or CLAUDE_API_KEY is missing, the app uses deterministic demo commentary instead of breaking.

Pvlseon

The hardest part was making the demo flexible without making it feel generic or overbuilt.

Challenge	What I changed	Trade-off	Learning
Different CSV schemas	Added auto-detection with manual mapping overrides.	A few more controls, but a much less brittle demo.	The data layer has to be solid before the charts matter.
API keys and reruns	Moved AI behind a Generate button and cached the result.	One extra click, but API usage is intentional.	A good demo should not surprise the reviewer in a bad way.
AI output quality	Used compact KPI context, JSON parsing, and fallback text.	Less free-form output, more predictable rendering.	Structure beats clever prompting.
Exporting charts	Converted Plotly figures into PDF images with Kaleido and ReportLab.	PDF prep takes a moment, but the final artifact is complete.	Deliverables need their own reliability path, not just screenshots.

Takeaway: I treated this like a small product, not a notebook. The app works, but just as important, the choices are explainable in a review.

Pvlseon

Pvlseon is ready as a focused demo, with a realistic path to production depth.

Final solution

- CSV upload and built-in demo datasets
- Editable column mapping for messy schemas
- KPI cards and monthly trend explorer
- Claude AI insights and a one-paragraph AI summary
- No-key fallback commentary for reviewers
- PDF export with cards, charts, and AI text

`streamlit run app.py`

Next steps

Product

Add saved dashboard configurations and clearer sample scenarios for each dataset.

Data

Support Excel files, database connectors, and KPI templates by business type.

AI

Let Claude compare segments, explain anomalies, and generate follow-up questions for leadership.

Repo link: https://github.com/jordanjasonclifford/smart_kpi_dashboard

Demo flow: pick or upload a dataset, confirm the mapping, review KPI movement, generate insights, then export the PDF summary.